

Design and Analysis of Algorithms (DAA)

Obfuscation & Deobfuscation: A Comprehensive Algorithmic Report

Subject: Design and Analysis of Algorithms

Domain: Cybersecurity — Code Obfuscation & Deobfuscation

Report Type: Project Ideation, Algorithm Mapping & Complexity Analysis

Table of Contents

1. Introduction
 2. Core DAA Concepts Involved
 3. Project 1 — CFG Flattening & Unflattening
 4. Project 2 — Polymorphic Code Detection via Sequence Alignment
 5. Project 3 — Opaque Predicate Injection & AST Simplification
 6. Project 4 — High-Entropy Packer Analysis via Sliding Windows
 7. Comparative Summary Table
 8. Theoretical Bounds & Hardness
 9. Recommendation & Conclusion
-

1. Introduction

Obfuscation and deobfuscation are two sides of the same algorithmic coin, deeply rooted in the core principles of the Design and Analysis of Algorithms. Obfuscation is the process of deliberately transforming code or data to make it harder to analyze — essentially, maximizing the complexity of reverse engineering. Deobfuscation is the inverse: applying systematic algorithms to undo that complexity and recover the original structure.

From a DAA perspective, this domain is exceptional because:

- **Obfuscation** maps to problems of **graph transformation**, **tree mutation**, and **combinatorial explosion** — pushing static analysis toward NP-Hard territory.
- **Deobfuscation** maps to **graph traversal** (DFS/BFS), **dynamic programming**, **greedy algorithms**, and **tree reduction** — all foundational DAA topics.
- Complexity analysis (time, space, worst-case, average-case) can be rigorously applied to every technique.

The four project ideas below each represent a distinct cluster of DAA algorithms applied to cybersecurity.

2. Core DAA Concepts Involved

DAA Concept	Where It Appears
Graph Theory (DFS, BFS, Dominator Trees)	CFG Flattening / Unflattening
Dynamic Programming (DP)	Sequence Alignment (Needleman-Wunsch, LCS)
Greedy Algorithms	Entropy sliding window optimization
Tree Traversal (Post-order, In-order)	AST simplification, constant folding
String Matching (KMP, Rabin-Karp)	Polymorphic pattern detection baseline
Sliding Window Technique	Entropy-based packer analysis
NP-Completeness (SAT, Boolean Logic)	Opaque predicate theoretical limits
Divide and Conquer	Recursive AST decomposition
Heuristics & Approximation Algorithms	Real-world deobfuscation

3. Project 1 — Control Flow Graph (CFG) Flattening & Unflattening

3.1 Concept

Control Flow Flattening is a powerful obfuscation technique that dismantles the natural hierarchical flow of a program. It takes structured constructs (if/else, loops, function calls) and collapses them into a single flat dispatcher — typically a `switch` statement inside an infinite loop. Each basic block of the program becomes one `case` in the switch, and a **state variable** determines which block executes next.

This is a **graph transformation problem**: the original directed graph (CFG) is restructured to destroy recognizable patterns while preserving semantic equivalence.

3.2 DAA Algorithm Mapping

Obfuscation Phase — Graph Transformation

Input: A directed graph $G = (V, E)$, where V = basic blocks, E = control flow edges.

Algorithm:

1. Perform a **topological sort** of G to determine a valid execution order — $O(V + E)$.
2. Assign a unique integer state to each node (basic block).
3. Create a new "dispatcher" node D connected to all other nodes.
4. Replace all direct edges ($u \rightarrow v$) with:
 - u sets state variable $s = \text{state}(v)$
 - D reads s and jumps to v
5. Output: A new graph $G' = (V \cup \{D\}, E')$ where every node connects only through D .

Complexity:

- Time: $O(V + E)$ for topological sort + $O(V)$ for state assignment = $O(V + E)$
- Space: $O(V)$ for state mappings + $O(E)$ for the new adjacency structure = $O(V + E)$

Deobfuscation Phase — Graph Analysis and Reconstruction

Step 1 — Identify the Dispatcher Node

Use **in-degree analysis**. The dispatcher has in-degree $\approx V$ (almost all nodes point back to it).

Complexity: $O(V + E)$

Step 2 — Trace Execution Paths (DFS-based)

Run DFS from each possible entry point, tracking state variable transitions.

Reconstruct original edges by observing which state value each block assigns before returning to dispatcher.

Complexity: $O(V + E)$ per DFS traversal

Step 3 — Build Dominator Tree

Use the **Lengauer-Tarjan Algorithm** to compute the dominator tree of the flattened graph.

A node u **dominates** node v if every path from the entry to v passes through u .

This reveals the original hierarchical structure of the program.

Complexity: $O((V + E) \log V)$ — near-linear, optimal for sparse graphs.

Step 4 — Reconstruct Original CFG

Prune the dispatcher node and its artificial edges. Re-establish direct edges based on state-variable analysis.

Complexity: $O(V + E)$

Overall Deobfuscation Complexity: $O((V + E) \log V)$

3.3 Complexity Analysis Summary

Phase	Algorithm	Time Complexity	Space Complexity
Obfuscation	Topological Sort + Graph Rebuild	$O(V + E)$	$O(V + E)$
Dispatcher Identification	In-degree scan	$O(V + E)$	$O(V)$
Path Tracing	DFS	$O(V + E)$	$O(V)$
Dominator Tree	Lengauer-Tarjan	$O((V+E) \log V)$	$O(V)$
CFG Reconstruction	Edge pruning	$O(V + E)$	$O(E)$

3.4 Visualization / Experiment Design

- Generate random programs with $N = \{10, 50, 100, 500\}$ basic blocks.
- Measure time to flatten and unflatten as N grows.
- Plot: **Time vs. $V+E$** to empirically verify $O((V+E) \log V)$.
- Metric: Number of edges added (bloat factor = $|E'| / |E|$).

4. Project 2 — Polymorphic Code Detection via Sequence Alignment

4.1 Concept

Polymorphic malware uses a mutation engine to constantly rewrite its own bytecode while preserving its functionality. Common mutations include:

- Register substitution (`mov eax`) → (`mov ecx`)
- Junk instruction insertion (`nop`), (`xor reg, reg`)
- Instruction reordering (for independent operations)
- Equivalent instruction substitution (`add eax, 1`) → (`inc eax`)

Traditional signature-based detection (exact string matching) fails because the byte sequence changes every time. The solution is to treat malware detection as a **biological sequence alignment problem** — measuring similarity rather than identity.

4.2 DAA Algorithm Mapping

Obfuscation Phase — Mutation Engine

Input: A sequence of instruction tokens $I = [i_1, i_2, \dots, i_n]$

Algorithm: For each instruction i_k , randomly apply one of:

- **Register swap:** Replace register name using a lookup table — $O(1)$ per instruction
- **Junk injection:** Insert a random nop-equivalent instruction — $O(1)$ per instruction
- **Reorder:** Swap two adjacent independent instructions — $O(1)$ per swap (after dependence check $O(n)$)

Total Mutation Complexity: $O(n)$ where n = number of instructions.

Deobfuscation Phase — Sequence Alignment

Baseline Failure: Exact String Matching

- KMP (Knuth-Morris-Pratt): $O(n + m)$ — fails on any single mutation
- Rabin-Karp: $O(n + m)$ average — fails similarly

Both are $O(n+m)$ but **ineffective** for polymorphic variants, motivating the DP approach.

Algorithm 1: Longest Common Subsequence (LCS) — Dynamic Programming

Find the longest sequence of instructions common between the known malware signature S and the mutated sample M .

Recurrence:

```
LCS[i][j] = LCS[i-1][j-1] + 1           if S[i] == M[j]
LCS[i][j] = max(LCS[i-1][j], LCS[i][j-1]) otherwise
```

Similarity Score = $\text{LCS}(S, M) / \max(|S|, |M|)$

If score > threshold $\theta \rightarrow$ malware detected.

- Time Complexity: $O(m \times n)$
- Space Complexity: $O(m \times n)$ for full DP table; reducible to $O(\min(m, n))$ with space optimization.

Algorithm 2: Needleman-Wunsch — Global Alignment (DP)

A more sophisticated DP algorithm that introduces gaps (for junk insertions/deletions) and a substitution score matrix (for register swaps).

Recurrence:

```

NW[i][j] = max(
    NW[i-1][j-1] + score(S[i], M[j]),    // match/mismatch
    NW[i-1][j]   + gap_penalty,          // gap in M
    NW[i][j-1]   + gap_penalty           // gap in S
)

```

- Time Complexity: $O(m \times n)$
- Space Complexity: $O(m \times n)$ (traceback matrix needed for alignment)

Algorithm 3: Smith-Waterman — Local Alignment

Finds the best-matching *subsequence* rather than full alignment. Better suited for detecting partial code reuse (partial polymorphism).

- Time Complexity: $O(m \times n)$
- Space Complexity: $O(m \times n)$ or $O(n)$ with Gotoh's space optimization.

4.3 Complexity Comparison

Algorithm	Detection Model	Time	Space	Handles Junk Code
KMP	Exact match	$O(n + m)$	$O(m)$	No
Rabin-Karp	Exact hash	$O(n + m)$ avg	$O(1)$	No
LCS (DP)	Subsequence similarity	$O(m \times n)$	$O(m \times n) \rightarrow O(n)$	Partially
Needleman-Wunsch	Global alignment	$O(m \times n)$	$O(m \times n)$	Yes (gap penalty)
Smith-Waterman	Local alignment	$O(m \times n)$	$O(m \times n)$	Yes

4.4 Experiment Design

- Generate a base malware signature of length $m = 100$ tokens.
- Apply increasing levels of mutation (0%, 10%, 25%, 50%, 75%) to create test samples.
- Measure similarity scores from LCS and NW across mutation levels.
- Plot: **Similarity score vs. Mutation %** to show detection threshold behavior.
- Measure: **Runtime vs. sequence length** to empirically verify $O(m \times n)$.

5. Project 3 — Opaque Predicate Injection & AST Simplification

5.1 Concept

An **opaque predicate** is a conditional expression whose outcome (always-true or always-false) is mathematically certain but computationally expensive to determine through static analysis.

Examples:

- $\text{if } (x^2 + x) \% 2 == 0$ — always TRUE for any integer (since $x^2 + x = x(x+1)$ is always even)
- $\text{if } (p * (p+1) * (p-1) \% 6 == 0)$ — always TRUE (product of 3 consecutive integers divisible by 6)
- Graph-based: $\text{if } (3\text{-colorable}(G))$ where G is a pre-chosen non-3-colorable graph

These are injected into the Abstract Syntax Tree (AST) of the code, bloating it with unreachable dead code that confuses human readers and static analyzers.

5.2 DAA Algorithm Mapping

Obfuscation Phase — AST Injection

Input: A program's AST (a rooted directed tree)

Algorithm:

1. Select a set of injection points (leaf nodes or edges in the AST) — $O(V)$ scan.
2. For each injection point, generate an opaque predicate expression subtree — $O(\text{depth of predicate})$.
3. Attach the new subtree as a conditional node with dead-code branches.

Complexity: $O(V \times D)$ where V = AST nodes, D = depth of predicate expression.

Deobfuscation Phase — AST Simplification (Tree Reduction)

Step 1 — Post-order Traversal for Constant Folding

Traverse the AST in post-order (left $\rightarrow$ right $\rightarrow$ root). At each node, if both children are constants, evaluate the expression and replace the subtree with a single constant leaf.

Example: $3 * 4 + 2 \rightarrow \text{node } (+, (*, 3, 4), 2) \rightarrow (+, 12, 2) \rightarrow 14$

Complexity: $O(V)$ — one pass through all V nodes.

Step 2 — Dead Code Elimination

After constant folding, any conditional node $\text{if } (\text{constant})$ can be evaluated:

- `if (TRUE) { A } else { B }` → replace with just A
- `if (FALSE) { A } else { B }` → replace with just B

Prune the unreachable branch subtree.

Complexity: $O(V)$ — each node visited once.

Step 3 — Algebraic Identity Simplification (Tree Rewriting)

Apply known algebraic simplifications:

- `x + 0 → x`
- `x * 1 → x`
- `x - x → 0`
- `x % 2 == 0` where $x = n(n+1) \rightarrow \text{TRUE}$

This is a **pattern-matching on tree structure** problem.

Complexity: $O(V \times P)$ where P = number of rewriting rules. In practice, P is fixed, so $O(V)$.

Step 4 — Theoretical Limit: SAT Reduction

Not all opaque predicates can be detected. Determining whether an arbitrary predicate is always-true or always-false is equivalent to checking **Boolean Satisfiability (SAT)**, which is **NP-Complete**.

This establishes the theoretical lower bound: no polynomial-time algorithm can fully deobfuscate all opaque predicates in the general case.

5.3 Complexity Summary

Phase	Algorithm	Time	Space
Predicate Injection	AST subtree insertion	$O(V \times D)$	$O(D)$ per predicate
Constant Folding	Post-order traversal	$O(V)$	$O(h)$ — stack depth
Dead Code Elimination	Conditional pruning	$O(V)$	$O(1)$ per node
Algebraic Simplification	Tree rewriting	$O(V \times P) \approx O(V)$	$O(1)$
General predicate evaluation	SAT (theoretical)	NP-Complete	—

5.4 Experiment Design

- Build a simple expression evaluator with AST representation.

- Inject K opaque predicates ($K = 1, 5, 10, 20, 50$) into a base program.
 - Measure time for the simplifier to reduce the bloated AST.
 - Plot: **AST size before vs. after simplification** and **time vs. K**.
 - Demonstrate: cases where the simplifier succeeds (polynomial) vs. cases that reduce to SAT (intractable).
-

6. Project 4 — High-Entropy Packer Analysis via Sliding Windows

6.1 Concept

Packers and encryptors are used to conceal malicious code inside a binary. They work by encrypting or compressing the payload, storing it as encrypted bytes in the binary, and using a small "stub" to decrypt it at runtime. Packed sections appear as **high-entropy random data** compared to normal executable code, which has structured, low-entropy patterns.

Shannon Entropy is the mathematical measure of randomness:

$$H(X) = -\sum p(x_i) \times \log_2(p(x_i))$$

For a uniform random byte distribution (fully encrypted): $H \approx 8.0$ bits

For typical code/text: $H \approx 4.5$ – 6.5 bits

The detection task is a **scanning and optimization problem**: efficiently locate high-entropy regions in a potentially large binary.

6.2 DAA Algorithm Mapping

Obfuscation Phase — Rolling XOR Cipher (Packing)

Algorithm:

```
key = random byte k
for each byte  $b_i$  in payload:
    encrypted[i] =  $b_i$  XOR k XOR i    // rolling key
```

Complexity: $O(N)$ — one pass through N bytes.

Deobfuscation Phase — Entropy Scanning

Naive Approach: Recompute entropy for every window

For a window of size W at position i:

1. Count frequency of each byte value (256 possible values) — $O(W)$
2. Compute H using the formula — $O(256) = O(1)$
3. Slide one position: repeat from step 1

Total: $O(N \times W)$ — **inefficient** for large W .

Optimized Approach: Sliding Window Algorithm

Key insight: When the window slides from position i to $i+1$:

- Only one byte is **removed** (position i)
- Only one byte is **added** (position $i+W$)

Maintain a frequency table of 256 buckets. Update incrementally:

```
Algorithm SlidingEntropyWindow(file, W):
    Build freq[] for first window of size W          //  $O(W)$ 
    Compute  $H_0$  from freq[]                          //  $O(256)$ 

    for i = 1 to N - W:
        outgoing_byte = file[i-1]
        incoming_byte = file[i+W-1]

        Update freq[outgoing_byte] -= 1             //  $O(1)$ 
        Update freq[incoming_byte] += 1             //  $O(1)$ 
        Update H incrementally                       //  $O(1)$  – update only changed
        terms

        if H > threshold:
            flag region [i, i+W] as packed           //  $O(1)$ 

    Total:  $O(W) + O(N) = O(N)$ 
```

Incremental entropy update formula:

```
H_new = H_old
        + p_old(out) × log2(p_old(out))    - p_new(out) × log2(p_new(out))
        + p_old(in)  × log2(p_old(in))    - p_new(in)  × log2(p_new(in))
```

Only 2 terms change per step → **$O(1)$** per window slide.

Frequency Analysis for Cipher Breaking

After locating a packed region:

- 1. Collect byte frequency histogram — $O(W)$
- 2. Sort by frequency (most common byte XOR'd with most common plaintext byte = key) — $O(256 \log 256) = O(1)$
- 3. Test candidate key against known plaintext patterns — $O(W)$

Complexity: $O(W)$ per packed region.

6.3 Complexity Comparison

Approach	Time Complexity	Space Complexity	Notes
Naive (recompute per window)	$O(N \times W)$	$O(256) = O(1)$	Infeasible for large files
Sliding Window (incremental)	$O(N)$	$O(256) = O(1)$	Optimal — single pass
Frequency Analysis (cipher)	$O(W)$	$O(256) = O(1)$	Per detected region

6.4 Experiment Design

- Create synthetic binary files of size $N = \{1\text{KB}, 10\text{KB}, 100\text{KB}, 1\text{MB}\}$ with embedded "packed" regions of varying sizes.
- Compare naive $O(N \times W)$ vs. sliding window $O(N)$ implementations.
- Plot: **Runtime vs. File Size** for both approaches.
- Calculate speedup factor = $(N \times W) / N = W$ — confirm linear improvement with W .
- Measure detection accuracy (precision/recall) at different entropy threshold values.

7. Comparative Summary Table

Project	Obfuscation Technique	Core DAA Algorithms	Best Time Complexity	Space Complexity	Hardness
1. CFG Flattening	Graph restructuring	DFS, Dominator Trees, Topo Sort	$O((V+E) \log V)$	$O(V + E)$	Polynomial
2. Polymorphic Detection	Instruction mutation	LCS (DP), Needleman-Wunsch	$O(m \times n)$	$O(m \times n) \rightarrow O(n)$	Polynomial

Project	Obfuscation Technique	Core DAA Algorithms	Best Time Complexity	Space Complexity	Hardness
3. Opaque Predicates	AST injection	Post-order Traversal, Constant Folding	$O(V)$ (approx.)	$O(h)$	NP-Complete (general)
4. Packer Analysis	XOR encryption	Sliding Window, Shannon Entropy	$O(N)$	$O(1)$	Polynomial

8. Theoretical Bounds & Hardness

8.1 Why Deobfuscation Cannot Always Be Polynomial

The **halting problem** and **Rice's theorem** tell us that any non-trivial semantic property of programs is undecidable in general. Obfuscation exploits this:

- **Opaque predicates** → Deciding if a predicate is always-true reduces to SAT → **NP-Complete**
- **CFG flattening with encrypted dispatchers** → Symbolic execution becomes exponential (path explosion)
- **Packing with unknown ciphers** → Cipher identification is NP-Hard in the general case

8.2 The Complexity Spectrum of Deobfuscation



8.3 Approximation and Heuristics

Real-world deobfuscation tools don't solve the NP-Hard cases exactly — they use:

- **Threshold-based heuristics:** Flag if entropy > 7.2 , rather than proving encryption
- **Bounded symbolic execution:** Explore only paths up to depth k
- **Probabilistic matching:** Flag if LCS similarity $> 70\%$, rather than proving identity

This connects to the broader DAA topic of **approximation algorithms**: trading exactness for tractability.

9. Recommendation & Conclusion

Recommended Project for DAA Grading

Primary Choice: CFG Flattening & Unflattening (Project 1)

- Covers graph theory (DFS, BFS, Dominator Trees) — 4+ core DAA topics
- Clear visualization (draw the original CFG vs. flattened CFG)
- Measurable metrics: node/edge counts, runtime, bloat factor
- Can demonstrate sub-problems: topological sort, DFS, in-degree analysis separately

Secondary Choice: Polymorphic Detection via Sequence Alignment (Project 2)

- Classic DP — LCS is a textbook DAA problem with direct application
- Easily implement and compare $O(n+m)$ string matching vs. $O(m \times n)$ DP
- Visualizable: similarity matrix, alignment paths

Mapping to DAA Grading Rubric

Rubric Criterion	How This Project Addresses It
Algorithm Design	Multiple algorithms designed (DFS, DP, sliding window)
Correctness Proof	Formal recurrences for DP; DFS correctness via loop invariants
Time Complexity Analysis	Big-O for every sub-problem
Space Complexity Analysis	Auxiliary space analysis for trees, matrices, frequency tables
Comparison of Approaches	Naive vs. optimized ($O(N \times W)$ vs. $O(N)$); exact vs. heuristic
Experimental Validation	Runtime plots confirming theoretical complexity
NP-Hardness Discussion	SAT reduction from opaque predicates

Conclusion

Obfuscation and deobfuscation are not just cybersecurity topics — they are living laboratories for the full spectrum of DAA. Graph algorithms, dynamic programming, tree traversal, sliding windows, and complexity theory all intersect naturally in this domain. A project in this space gives you both intellectual depth (NP-hardness discussions, Dominator Tree algorithms) and practical impact (real malware analysis techniques), making it an outstanding choice for a DAA course project.

Report compiled for DAA course project analysis. All complexity bounds reference standard computer science definitions (RAM model, worst-case unless stated otherwise).